FLUX

Image Generator

REST API Developer Guide

JavaScript

API v1 · built 2026-08-13

Contents

1 What this API is

2 Getting started

3 Authentication

4 Readiness and capabilities

5 Generating an image

6 Tracking progress

7 Editing with reference images

8 Importing references

9 Images and housekeeping

10 Vision-model jobs

11 Switching models

12 Handling errors

13 Endpoint reference

1 What this API is

The FLUX image generator runs FLUX.1 and FLUX.2 diffusion models on your own hardware and exposes them over HTTP on port 2222. This guide covers the REST API at /api/v1: generating images, tracking jobs, editing with reference images, driving the vision model, and managing which checkpoint is loaded.

Three properties shape every client you will write against it, and each one causes trouble if you assume otherwise.

Generation is always asynchronous

POST /jobs returns 201 with a job id immediately — even when the queue is empty. A single worker thread runs every job in turn, because the GPU can only serve one diffusion pipeline at a time. There is no synchronous "generate and return the image" call, and adding one would only hide the queue from you. Poll GET /jobs/{id} until `state` leaves `queued` and `running`.

Capabilities depend on the loaded model

The server runs one checkpoint at a time, chosen from a menu of 14 launcher configs. Negative prompts require the SDXL backend; inpainting requires FLUX.2 or SDXL; more than one reference image requires FLUX.2 or the Kontext editor. Sending an unsupported field is a 400, not a silent no-op, so read GET /model before you build a request rather than assuming.

Errors carry a stable code

Every failure is the same JSON shape, with the HTTP status carrying the category and a machine-readable `code` inside. Branch on the code. The message is written for a human reading a log and is free to change.

```
{"error": {"code": "queue_full",
           "message": "Queue is full (10 max). Cancel a queued job or wait."}}
```

2 Getting started

You need the server's API key — the same value it was started with, in FLUX_API_KEY or its .env file. Everything except the health check and the self-describing documents requires it.

```
export FLUX_API_KEY=your_key
export FLUX_URL=http://localhost:2222   # optional; this is the default
```

Confirm the server is up and see what it can do. This is the one endpoint that needs no key, so it is also how you check connectivity before you know whether your key is right.

```
curl -s localhost:2222/api/v1/health | jq
```

Install the client dependencies:

DEPENDENCIES

```
// No dependencies: the client is one ES module using fetch, which
// Node 18+ and every current browser already have.
//
// The sample client lives in examples/flux_client.js
FLUX_API_KEY=your_key node examples/flux_client.js health
```

The complete client used throughout this guide ships with the server. Everything below is an excerpt from it, so you can read the whole thing for context.

THE WHOLE THING, END TO END

```
import { FluxClient } from './flux_client.js';

const flux = new FluxClient({
    baseUrl: process.env.FLUX_URL ?? 'http://localhost:2222',
    apiKey: process.env.FLUX_API_KEY,
});
await flux.waitUntilReady();

const job = await flux.generate('a red fox in falling snow', { steps: 30 });
console.log(job.images[0].filename, 'seed', job.images[0].seed);
```

3 Authentication

Send the key as an X-API-Key header. An api_key query parameter works too, for consumers that cannot set headers — an <img> tag pointed at an image endpoint being the case it exists for. Prefer the header everywhere else: query strings end up in logs and browser history.

```
const res = await fetch('http://localhost:2222/api/v1/model', {
    headers: { 'X-API-Key': process.env.FLUX_API_KEY },
});
const info = await res.json();
console.log(info.description);

// The query-param form is what an <img> needs, since a tag sends no headers
img.src = `${base}/images/${filename}?api_key=${encodeURIComponent(key)}`;
```

> A missing or wrong key is 401 with code `unauthorized`. The key gates endpoints that read arbitrary server paths and fetch arbitrary URLs, so treat it as a host credential and do not expose the server to untrusted networks.

4 Readiness and capabilities

Loading a checkpoint takes anywhere from seconds to several minutes — a 32B FLUX.2 from cold is the slow end. Until it finishes, generation calls return 503 with code model_loading. GET /health reports progress and narrates what it is doing, which is worth surfacing rather than showing a spinner.

```
const state = await flux.health();
// { ready: false, status: 'loading FLUX.2 model', elapsed_s: 47.2 }

if (!state.ready) {
    await flux.waitUntilReady({ onStatus: s => console.log(`  ${s}...`) });
}
```

Once ready, ask what the loaded backend can actually do. Build your request from these flags rather than assuming — the same server binary serves very different capability sets depending on which config was launched.

```
const info = await flux.model();
if (info.inpaint) {
    // mask_image is accepted
}
if (info.negative_prompt) {
    // SDXL backend; negative_prompt is accepted
}
const maxRefs = info.kontext || info.flux_version === 2 ? 3 : 1;
```

GET /MODEL

Field	Meaning
`model, description`	Human-readable name of the loaded checkpoint
`flux_version`	1 or 2 — FLUX.2 accepts multiple references and masks
`negative_prompt`	SDXL backend; negative_prompt is accepted
`inpaint`	mask_image is accepted
`kontext`	Instruction-editing backend
`turbo, schnell`	Few-step variants — expect fewer steps than requested
`vae_tiling`	How the final decode is tiled: auto (the default — tile only above vae_tiling_threshold_mp), always, or off
`vae_tiling_threshold_mp`	Megapixel cutoff auto compares against (1.9 by default)

5 Generating an image

Only prompt is required. The call returns as soon as the job is queued; the helper below then polls until it settles and raises if it failed.

```
const job = await flux.generate('a red fox in falling snow, golden hour', {
    steps: 30,
    size: '1.5mp',
    orientation: 'widescreen',
});

for (const image of job.images) {
    console.log(image.filename, 'seed', image.seed);
    document.body.append(
        Object.assign(new Image(), { src: flux.imageUrl(image.filename) }));
}
```

POST /JOBS — REQUEST BODY

Field	Type	Default	Notes
`prompt`	string	—	Required
`steps`	int	25	1–200
`batch`	int	1	1–128; prompt encoded once for the batch
`seed`	int	random	Omit for a fresh seed per image
`guidance`	float	model	Non-negative
`strength`	float	0.5	0–1; img2img only
`orientation`	string	landscape	square, portrait, landscape, widescreen, extra-tall
`size`	string	1mp	0.25mp … 2mp
`negative_prompt`	string	—	SDXL only
`input_images`	string[]	[]	Base64 / data URLs, up to 3
`input_paths`	string[]	[]	Paths the server can read
`mask_image`	string	—	Inpainting; needs exactly one reference
`aspect_mode`	string	keep	keep derives dims from the reference
`show_preview`	bool	false	Decode latent previews while running
`save_previews`	bool	false	Also write frames to steps/

Asking for several images in one job is markedly cheaper than several jobs: the server encodes the prompt once and reuses the embedding for the whole batch, which matters because FLUX.2's text encoders are LLM-sized.

```
const job = await flux.generate('a lighthouse in a storm',
                                { batch: 8, steps: 28 });
console.log(job.images.length, 'images in', job.generation_time, 's');
```

6 Tracking progress

Submit and poll yourself when you want to render progress. A job moves queued → running → done | failed | canceled; while running, current/batch track which image is being produced and step/total_steps the diffusion progress.

```
const queued = await flux.submit('a mountain range at dawn',
                                  { steps: 40, show_preview: true });
console.log('queued at position', queued.position);

const job = await flux.waitForJob(queued.id, {
    onProgress: j => {
        if (j.state === 'running') {
            console.log(`image ${j.current}/${j.batch} `
                      + `step ${j.step}/${j.total_steps}`);
        }
    },
});
```

> total_steps is not always what you asked for. It is re-read from the live scheduler on the first step, because img2img, turbo and schnell variants denoise fewer steps than requested. Compute progress from the reported total, never from your own step count.

Intermediate images

A diffusion run passes through every intermediate state on its way to the final image, and you can watch or keep them. GET /jobs/{id}/previews returns both kinds at once.

live is the latent being denoised right now, decoded at reduced resolution and overwritten in place. It exists only while the job runs with show_preview, and because the file is reused, its ts changes on every frame — use the supplied url rather than caching the path. frames are per-step images written to disk, which additionally needs save_previews, but they persist after the job ends so a finished run can be replayed.

```
const queued = await flux.submit(prompt,
    { steps: 30, show_preview: true, save_previews: true });

for (;;) {
    const previews = await flux.previews(queued.id);
    if (previews.live) {
        const { step, total_steps, url } = previews.live;
        console.log(`step ${step}/${total_steps}  ${url}`);
        // `url` carries a cache-busting ts — the file is overwritten in place
        preview.src = url;
    }
    if (['done', 'failed', 'canceled'].includes(previews.state)) break;
    await new Promise(r => setTimeout(r, 1000));
}

// Kept frames outlive the job
for (const frame of (await flux.previews(queued.id)).frames) {
    console.log(frame.image, frame.step, frame.url);
}
```

> Previews cost real time: each one decodes a latent through the VAE. The server throttles live previews to at most one every 0.75s for that reason, but save_previews bypasses the throttle to capture every step — expect a measurably slower generation when you turn it on.

When you would rather watch than integrate, two endpoints render the same data as self-refreshing pages: /api/v1/queue.html and /api/v1/jobs/{id}/previews.html. A browser navigation cannot send a header, so pass the key in the query string.

WATCH IN A BROWSER

```
http://localhost:2222/api/v1/queue.html?api_key=YOUR_KEY
http://localhost:2222/api/v1/jobs/a1b2c3d4e5f6/previews.html?api_key=YOUR_KEY
```

Cancelling a queued job drops it; cancelling the running one interrupts the diffusion loop at the next step. Batch images that already finished are kept.

```
await flux.cancel(job.id);
```

Seeing the whole queue

One worker serves everyone, so your job may sit behind others. GET /queue answers where it is in line and roughly when it will run: each waiting entry carries a 1-based position, and estimated_wait_s extrapolates from recently completed jobs (null until at least one has finished).

```
const q = await flux.queue();
console.log(`${q.depth}/${q.capacity} waiting, `
          + `${q.images_pending} image(s) pending`);

for (const job of q.waiting) {
    console.log(job.position, job.id, job.prompt.slice(0, 50));
}

if (!q.accepting) {
    // a further submit would fail with queue_full
}
```

> `accepting` is the honest answer to "can I submit right now". It goes false once ten jobs are pending; the running job holds no pending slot, so a full queue can still have one generating.

7 Editing with reference images

Reference images travel as base64 data URLs in input_images. One reference works on every backend; two or three need FLUX.2 or Kontext. With aspect_mode "keep", the output dimensions are derived from the reference's aspect ratio at your requested megapixel count.

```
// toDataUrl takes anything File-like — a drop event, a file input, a Blob
const reference = await FluxClient.toDataUrl(input.files[0]);

const job = await flux.generate('make it winter, heavy snow on the roof', {
    input_images: [reference],
    strength: 0.55,
    aspect_mode: 'keep',
});
```

strength controls how far the result may drift from the reference: low values preserve the original closely, high values treat it as loose inspiration. It applies to FLUX.1 img2img; Kontext and FLUX.2 read the prompt as an edit instruction instead.

When the file already lives on the server, skip the upload entirely. input_paths accepts absolute paths, ~-prefixed paths, or paths relative to the output directory; the server loads them and folds them into input_images for you.

```
const job = await flux.generate('make it winter', {
    input_paths: ['archive/house.jpg'],
});
```

Inpainting

Supply exactly one reference plus a mask, on a FLUX.2 or SDXL backend. White in the mask marks the region to regenerate.

```
if ((await flux.model()).inpaint) {
    const job = await flux.generate('a brass telescope on the table', {
        input_images: [await FluxClient.toDataUrl(roomFile)],
        mask_image: await FluxClient.toDataUrl(maskFile),
    });
}
```

8 Importing references

Three endpoints turn something that is not yet a usable reference into one. All return the same shape — a JPEG data URL bounded to 2048px, plus its dimensions — so the result drops straight into input_images.

Endpoint	Takes	For
`POST /imports/raw`	multipart file	Camera RAW (NEF/DNG/CR3) a browser cannot decode
`POST /imports/url`	{"url"}	Remote images, fetched server-side with no CORS limits
`POST /imports/path`	{"path"}	Files on the server's own disk

```
const reference = await flux.importUrl('https://example.com/photo.jpg');
console.log(reference.width, reference.height);

const job = await flux.generate('in the style of a woodcut',
                                { input_images: [reference.image] });

// Camera RAW needs the rawpy package on the server (501 if absent).
// importRaw posts multipart, so it takes the File itself, not a data URL.
const raw = await flux.importRaw(rawInput.files[0]);
```

To find server-side paths, browse the filesystem. Relative directories resolve against the output folder; the response always carries an absolute dir and its parent so you can navigate without doing path arithmetic.

```
const listing = await flux.browse('archive');
for (const name of listing.dirs) console.log('[dir]', name);
for (const entry of listing.files) console.log(entry.filename);

// Entries carry only a filename; join them onto the absolute
// listing.dir to get a path the thumbnail endpoint can resolve.
grid.append(...listing.files.map(entry => Object.assign(new Image(),
    { src: flux.thumbnailUrl(`${listing.dir}/${entry.filename}`) })));
```

9 Images and housekeeping

Generated images are written to the server's output directory and named flux{1|2}_{date}_{time}_{hex}.png, each with a .prompt sidecar recording how it was made. The listing covers today's output, newest first.

```
for (const image of await flux.history()) {
    console.log(image.time, image.filename, image.prompt);
}

const blob = await flux.imageBlob(filename);   // a Blob, for canvas or download
img.src = flux.imageUrl(filename);             // or let the tag fetch it
```

Housekeeping has three levels. Saving copies an image somewhere the other two cannot reach; archiving moves the day's work aside non-destructively; deleting is permanent.

```
await flux.saveImage(filename);    // copy into .saved/, beyond archive and delete
await flux.archive();              // move today's output into archive/
await flux.deleteImage(filename);  // permanent
await flux.deleteToday();          // permanent, everything from today
```

> Both delete calls are irreversible and the delete-today form takes no confirmation. Archive first if there is any doubt.

10 Vision-model jobs

A local vision model (via ollama) backs three features. They share one resource, discriminated by task, because a single call can run for minutes — well past any sensible request timeout. POST returns 202 with an id; poll GET /vlm/jobs/{id}.

Task	Takes	Returns
`describe`	images[], think	A prompt that would recreate the photo
`boost`	prompt, level 1–5, has_image	The prompt rewritten in the model's idiom
`critique`	direction, ref_image, output_filename	A judgment plus a revised instruction

Photo to prompt

Hand it one to three images and get back a prompt describing them — with several, it composes a single scene combining them.

```
const prompt = await flux.describe(
    await FluxClient.toDataUrl(photoInput.files[0]));
console.log(prompt);

// then generate a fresh image from that description alone
const job = await flux.generate(prompt);
```

Improving a prompt

Boost rewrites a draft into the prompting idiom of whichever model is loaded — descriptive prose for FLUX, an imperative instruction for Kontext, tag phrases for SDXL. level runs 1 to 5, from polishing the wording to reimagining the idea. Pass has_image when references will be attached: the idiom shifts to edit instructions.

```
const result = await flux.boost('a castle', { level: 4 });
const job = await flux.generate(result.prompt);
```

> A vision-model failure polls as 502 with code `vlm_failed` — the request was fine, the model was not. In practice this means ollama is not running or the model has not been pulled. GET /telemetry reports its residency.

11 Switching models

The checkpoints are far too large to hot-swap in-process, so switching is a supervised restart: the server records the target config, exits with a status its supervisor recognises, and comes back on the new one. PUT /models/current answers 202 — the restart is scheduled, not finished.

```
for (const config of (await flux.models()).configs) {
    console.log(config.id, config.label);
}

await flux.switchModel(9);
await new Promise(r => setTimeout(r, 3000));   // let the old process exit
await flux.waitUntilReady();
```

> Expect connection failures while polling across a restart — the process genuinely is gone for a moment. Both sample clients treat transport errors during a readiness wait as "still restarting" rather than fatal. If GET /models reports switchable: false the server was started without its supervisor and cannot restart itself; you will get 400 no_supervisor.

Comparing models on one prompt

A multi-model run generates the same prompt on several configs in turn, sharing one seed so the outputs are genuinely comparable. Because each config change restarts the process, the run is file-backed and survives them. Text-to-image only, and at most one run at a time.

```
const run = await flux.multiRun('a red fox in snow', [9, 6, 1], { steps: 28 });
console.log('seed', run.seed);

// multiRunStatus resolves to null once the run is gone; the server is
// unreachable across each restart, so treat a transport error as 'still going'
for (;;) {
    const state = await flux.multiRunStatus().catch(() => ({ active: true }));
    if (!state?.active) break;
    await new Promise(r => setTimeout(r, 5000));
}
```

12 Handling errors

Match on the code. These are the ones worth handling distinctly: model_loading and queue_full are both temporary and worth retrying, busy means something else must finish first, and invalid_request means the request will never succeed as written.

Code	Status	Meaning
`unauthorized`	401	Missing or wrong API key
`invalid_request`	400	A parameter failed validation
`not_found`	404	No such job, image, run, or path
`busy`	409	Jobs in flight block this operation
`queue_full`	429	Ten jobs already pending
`model_loading`	503	Model not up yet — poll /health
`no_supervisor`	400	Server cannot restart itself
`undecodable_image`	400	A reference or mask could not be decoded
`fetch_failed`	400	A remote URL could not be retrieved
`too_large`	400/413	Over the 64MB cap
`vlm_failed`	502	The vision model failed
`io_error`	500	A filesystem operation failed

```
import { FluxError } from './flux_client.js';

try {
    const job = await flux.generate(prompt);
} catch (e) {
    if (!(e instanceof FluxError)) throw e;   // transport, not a rejection
    if (e.code === 'queue_full') {
        await new Promise(r => setTimeout(r, 30_000));   // transient — retry
    } else if (e.code === 'model_loading') {
        await flux.waitUntilReady();
    } else {
        throw e;                              // including invalid_request
    }
}
```

Transport failures are a separate category from rejections, and both clients keep them distinct. During a model switch the server is legitimately absent for a while, so a connection error there means "wait", not "fail".

13 Endpoint reference

The complete surface. A running server serves this as a browsable page at /api/v1/docs and as an OpenAPI 3.1 document at /api/v1/openapi.json, either of which can generate a client for a language not covered here.

Method	Path	Purpose
GET	/api/v1	Service index (public)
GET	/api/v1/health	Readiness (public)
GET	/api/v1/openapi.json	OpenAPI document (public)
GET	/api/v1/docs	Browsable reference (public)
GET	/api/v1/model	Loaded backend capabilities
GET	/api/v1/models	Available configs
GET	/api/v1/models/current	Running config
PUT	/api/v1/models/current	Switch config (restarts)
GET	/api/v1/telemetry	GPU power, vision-model residency
POST	/api/v1/jobs	Queue a generation job
GET	/api/v1/queue	Ordered queue with wait estimate
GET	/api/v1/queue.html	Live HTML view of the queue
GET	/api/v1/jobs	Running, queued, recent jobs
GET	/api/v1/jobs/{id}	One job
DELETE	/api/v1/jobs/{id}	Cancel or interrupt
GET	/api/v1/jobs/{id}/previews	Intermediate images
GET	/api/v1/jobs/{id}/previews.html	Live HTML view of those
DELETE	/api/v1/jobs/recent	Clear finished list
GET	/api/v1/images	Today's images
GET	/api/v1/images/{name}	Image bytes
DELETE	/api/v1/images/{name}	Delete one
DELETE	/api/v1/images	Delete all of today's
POST	/api/v1/images/{name}/save	Preserve in .saved/
POST	/api/v1/archive	Archive today's output
POST	/api/v1/filmstrips	Compose an edit-loop strip
POST	/api/v1/imports/raw	Camera RAW to reference
POST	/api/v1/imports/url	Remote image to reference
POST	/api/v1/imports/path	Server file to reference
GET	/api/v1/files	Browse server directories

Method	Path	Purpose
GET	/api/v1/files/thumbnail	Thumbnail of a server image
POST	/api/v1/vlm/jobs	Start describe/boost/critique
GET	/api/v1/vlm/jobs/{id}	Poll a vision job
POST	/api/v1/multi-runs	Run one prompt on many configs
GET	/api/v1/multi-runs/current	Active run
DELETE	/api/v1/multi-runs/current	Cancel or dismiss